GaN Quantum Dot Detection and Analysis

Project Overview and Current Methodology

Poline

August 15, 2026

Contents

1 Project Overview

The purpose of this project is to develop and evaluate an automated pipeline for the detection and quantitative characterisation of semiconductor quantum dots (QDs) from atomic force microscopy (AFM) images.

Analysis pipeline is as follows:

1. convert the original AFM data into a convenient numerical representation;
2. remove scan tilt, line artefacts and slowly varying background structure;
3. identify candidate quantum dots;
4. estimate physical QD parameters such as position, height, radius, area and FWHM;
5. compare automatically obtained results against manually curated ground truth;
6. compare classical image-processing algorithms with machine-learning approaches.
7. optimise the final selected algorithm for the best F1 score.

2 Repository Structure

The project is divided into a small number of sections.

File / directory	**Purpose**
`Data-Conversion/`	Conversion of raw AFM data into NumPy arrays used by the analysis code. It also stores all unlabelled AFM data. .spm → .gwy → .npy.
`Image-Preprocessing/`	Artefact removal so that it is easier to detect QDs.
`Ground-Truth/`	Stores all labelled datasets used for model training, model itself and height/ FWHM manual data from Xin. Also contains app created to label QDs (heavily inspired by Cobi's)
`detection_algorithms.py`	Implementation of the classical QD detection algorithms using a common output structure. All have the same output format.
`detection_errors.py`	Functions for quantitative comparison of predicted and ground-truth detections.
`QD-gui.py`	Interactive interface for viewing and analysing QDs in AFM scans with an option to select multiple detection algorithms, different thresholds and kernels for Top Hat.
`method-COMPARISON.ipynb`	Notebook used for comparison of different QD detection approaches. Contains all of the information about model's and algorithms' performance and overlays errors, false positives and negatives of the algorithms based on the ground truth dataset.
`npy-data-QD-DETECTION.ipynb`	Experimental notebook for applying QD detection methods to NumPy AFM data. Used to summarise what I learned about image processing and demonstrate why certain algorithms have been chosen.
`npy-data-QD-ANALYSIS.ipynb`	Notebook for subsequent analysis of detected quantum dots.
`npy-data-TRANSFORMS.ipynb`	Notebook used for testing and visualising transformations applied to AFM images.

Table 1: High-level organisation of the repository.

3 AFM Data Conversion

3.1 `Data-Conversion/spm-to-gwy.py`

The original microscope measurements are stored in SPM files. The first conversion stage handles these microscope-specific files and converts them into Gwyddion-compatible data.

Using an intermediate Gwyddion representation is useful because Gwyddion understands the structure and metadata of common AFM formats and provides a convenient bridge between the proprietary microscope format and ordinary numerical arrays.

3.2 `Data-Conversion/gwy-to-npy.py`

The second conversion stage reads Gwyddion files and searches recursively for two-dimensional data fields corresponding to image channels.

Each detected two-dimensional channel is converted to a NumPy array and saved as a separate `.npy` file.

The resulting arrays are the principal numerical input used throughout the rest of the project.

4 Image Preprocessing

AFM scans contain structure unrelated to the quantum dots themselves. Examples include sample tilt, scanner-induced line offsets, stripes and slowly varying background curvature. Instead of having to open Gwyddion everytime to preprocess the image, the same can be done in either of the GUIs.

Direct application of a detection algorithm to the raw image can therefore produce incorrect QD positions and measurements.

The preprocessing code is divided into two files in order to not mix usual and more advanced processes for dealing with artefacts and show ideas development in a more orderly way.

4.1 `basic_data_preprocessing.py`

This file contains the main general-purpose AFM preprocessing operations.

4.1.1 Robust normalisation

Images can be rescaled using a robust estimate based on the median and median absolute deviation rather than the ordinary mean and standard deviation.

This reduces influence of unusually high QD peaks when estimating image typical scale.

4.1.2 Global plane removal

Large-scale sample tilt is modelled as

$$z(x, y) = ax + by + c.$$

The fitted plane is subtracted from the AFM image. The plane fit uses iterative outlier rejection so that strong foreground features have less influence on the estimated background.

4.1.3 Scan-line flattening

AFM images are acquired line-by-line. Individual scan lines may contain their own offsets or gradients. Each line can therefore be fitted independently using a low-order polynomial, which is then removed.

$$p(x) = \sum_{k=0}^{d} a_k x^k,$$

Robust fitting is again used to reduce the effect of quantum-dot peaks on the fitted background.

4.1.4 Scan-line alignment

Even after flattening, neighbouring scan lines may exhibit abrupt vertical offsets. The median value of each row is estimated and compared with a smoothed trend. Rapid changes in row height can then be removed while retaining slower physical variation.

4.1.5 Combined preprocessing pipeline

The standard preprocessing function currently combines the principal operations in sequence:

$$\text{raw image} \rightarrow \text{plane removal} \rightarrow \text{line flattening} \rightarrow \text{row alignment}.$$

Though, when labelling the data I only used line fitting so that less data is modified (though, correlation between raw image and 'destriped image' ame out to be 99 percent).

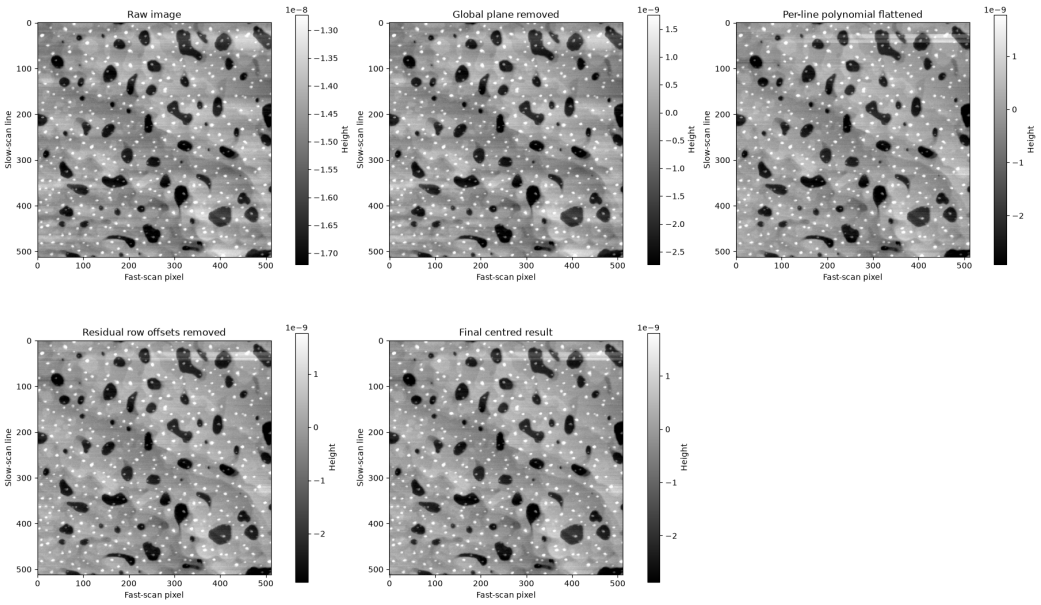

Figure 1: Foreground-aware preprocessing of the AFM image.

4.2 `foreground_preprocessing.py`

A limitation of ordinary background fitting is that quantum dots themselves may influence the fitted surface.

If a polynomial is fitted to every pixel, large QD peaks can pull the estimated background upward. Subtracting this background may then remove part of the physical QD height that is actually of interest.

The foreground-aware method addresses this problem iteratively:

1. fit an initial polynomial background;
2. calculate the residual image;
3. identify sufficiently large positive residuals as probable foreground;

4. dilate the foreground mask to exclude QD edges;
5. refit the polynomial using only probable background pixels;
6. repeat the procedure;
7. subtract the final fitted background.

This is intended to provide a stronger background-removal method for AFM images containing convex objects such as quantum dots.

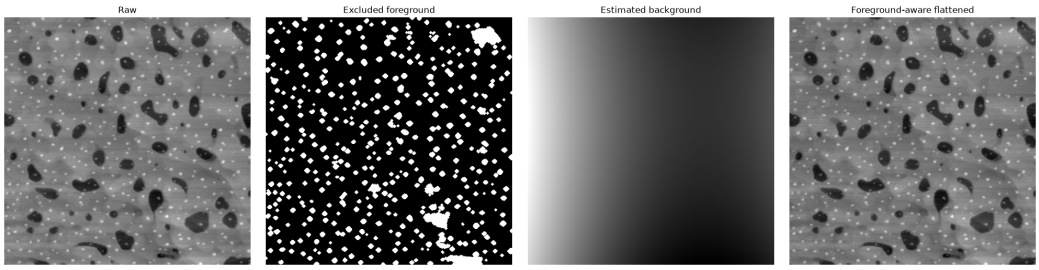

Figure 2: Foreground-aware preprocessing of the AFM image.

5 Ground-Truth Construction

The `Ground-Truth/` directory contains the manually curated reference data used to evaluate the algorithms and train the machine-learning model.

5.1 `NPY-Ground-Truth/`

This directory contains paired AFM images and binary masks.

For a scan,

- `channel_00___0_data.npy` contains the AFM height image;
- `channel_00___0_data_mask.npy` contains the corresponding ground-truth QD mask;
- feature CSV files can store measured properties of individual dots.

The binary masks are used both for algorithm evaluation and supervised machine-learning training.

5.2 `QD-gui-npy-TRAINING-SETS.py`

The ground-truth GUI provides an interactive method for constructing and correcting QD masks.

Initial QD candidates are generated automatically, after which detections can be manually corrected.

The editor supports operations such as removal of false-positive detections, ddition of false-negative QDs, local QD-height measurement, FWHM measurement, export of corrected masks, export of QD feature information to CSV files.

This allows the training and evaluation data to be manually inspected instead of assuming that an automatically generated segmentation is correct.

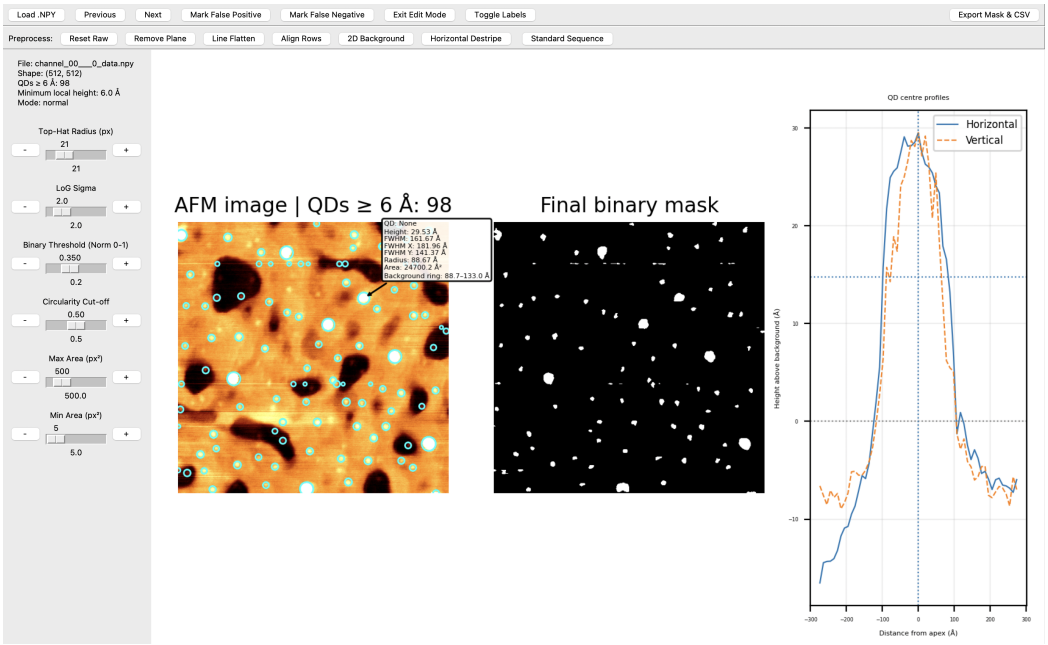

Figure 3: Foreground-aware preprocessing of the AFM image.

5.3 `initiation-tests.py`

This is a small diagnostic script. It checks that image and mask files are loaded correctly and reports properties such as image dimensions, data types, number of QD pixels and number of connected QD regions.

Its purpose is primarily dataset verification rather than QD detection itself.

6 Classical Quantum-Dot Detection

The principal classical detection methods are collected in

`detection_algorithms.py`.

A common output representation is used so that algorithms with very different internal mechanisms can still be compared consistently.

Typical outputs include a binary detection mask, QD centre coordinates, measured local heights, estimated radii, estimated areas.

Results are retained before and after application of the physical minimum-height criterion.

6.1 Laplacian of Gaussian

The Laplacian-of-Gaussian method searches for approximately blob-shaped structures across a range of spatial scales.

The detected LoG scale can be converted to an approximate QD radius.

This provides a simple scale-space baseline for comparison against more complicated methods.

6.2 Otsu-based detection

Threshold-based approaches determine candidate foreground pixels from the image-response distribution.

Otsu's method provides an automatically selected threshold rather than requiring one fixed value to be chosen manually for every scan.

6.3 Anisotropic diffusion

Denoising can be performed before blob detection while attempting to suppress noise without destroying important feature boundaries.

Both Chambolle-based smoothing and Perona–Malik-style anisotropic diffusion have been investigated.

6.4 Hough transform

Circular Hough transforms search explicitly for approximately circular structures over a selected range of radii.

This introduces stronger geometric assumptions than LoG detection.

6.5 Phase symmetry

Phase-based detection attempts to identify locally symmetric image structures.

Because it depends on local phase information rather than absolute intensity alone, it provides a qualitatively different approach to identifying QD-like objects.

6.6 SIFT

Scale-invariant feature detection has also been investigated as a possible method for identifying local QD structures over a restricted scale range.

6.7 Active contours

Geometric active contours and Chan–Vese segmentation evolve initial regions towards image boundaries.

These methods can be used to refine an existing set of candidate QD locations rather than detecting every QD entirely from scratch.

6.8 Watershed

The watershed implementation first enhances small bright structures using a white top-hat transformation.

After thresholding, a distance transform is calculated and local maxima are used as markers.

Watershed segmentation then separates neighbouring candidate regions.

6.9 Gabor filters

A bank of Gabor filters evaluates the image at several wavelengths and orientations.

The combined response can highlight spatial structures that resemble QDs while responding differently to directional scan artefacts.

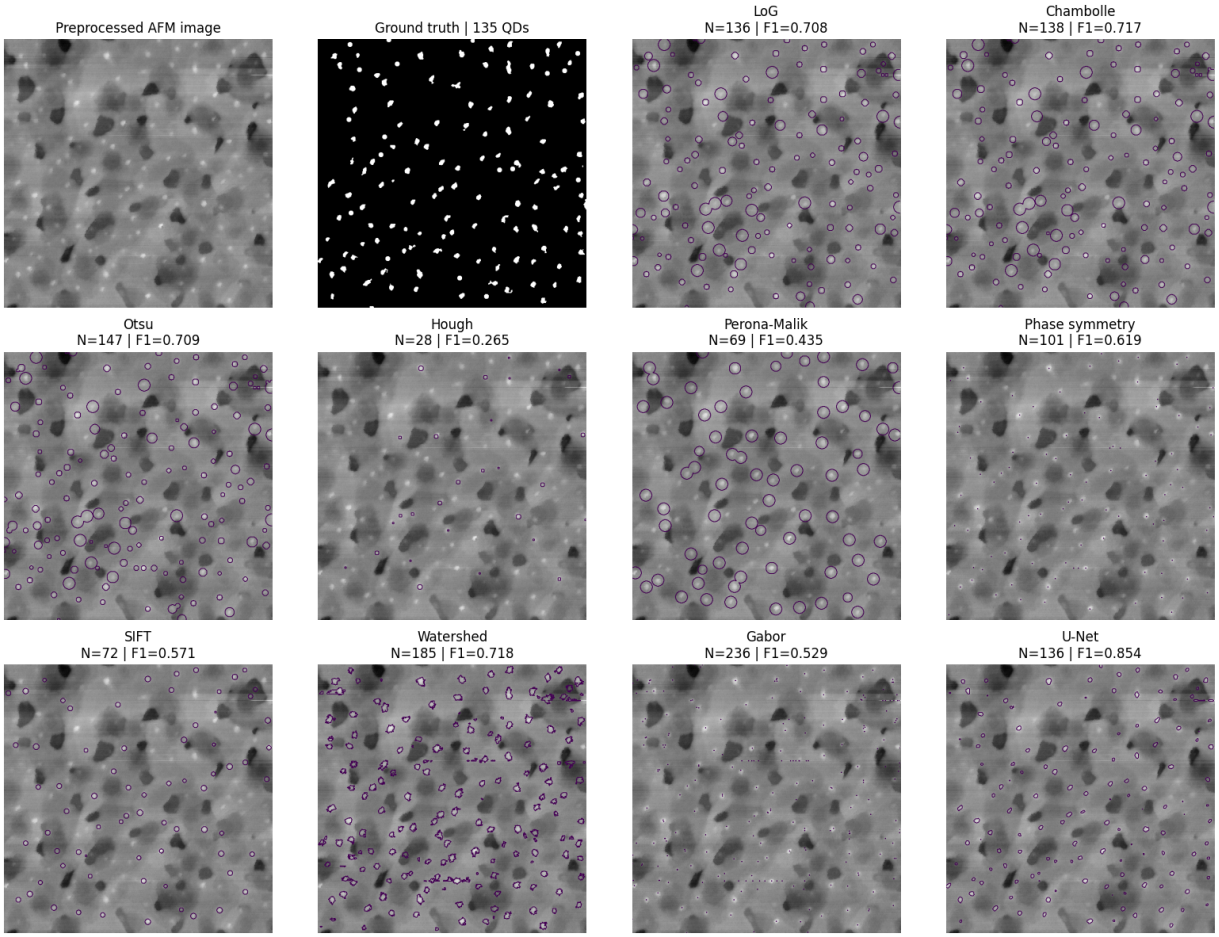

Figure 4: Foreground-aware preprocessing of the AFM image.

7 Physical QD Acceptance Criterion

Detection based only on image shape can identify small fluctuations that are not physically meaningful QDs.

For this reason, candidate detections can be subjected to a minimum local-height requirement.

The current criterion is

$$h_{\mathrm{QD}} \geq 6\,\mathring{A}.$$

Local QD height is measured relative to a surrounding estimate of the local background rather than relative to one global image level.

8 Detection Evaluation

Evaluation utilities are primarily contained in

`detection_errors.py`

and

`Ground-Truth/feature_evaluation.py`.

The project evaluates detection at both the pixel and object levels.

8.1 Pixel-level metrics

For a predicted binary mask and ground-truth mask, standard segmentation measures include precision, recall, F1 score, Dice coefficient and intersection over union.

The Dice coefficient is

$$D = \frac{2|P \cap T|}{|P| + |T|},$$

where P is the predicted QD mask and T is the true mask.

The intersection over union is

$$\mathrm{IoU} = \frac{|P \cap T|}{|P \cup T|}.$$

8.2 Object-level metrics

Pixel agreement does not completely describe QD detection performance.

The detected QD centres are therefore also compared with true QD centres using a spatial matching tolerance.

This provides

- true positives;
- false positives;
- false negatives;
- object precision;
- object recall;
- object F1 score.

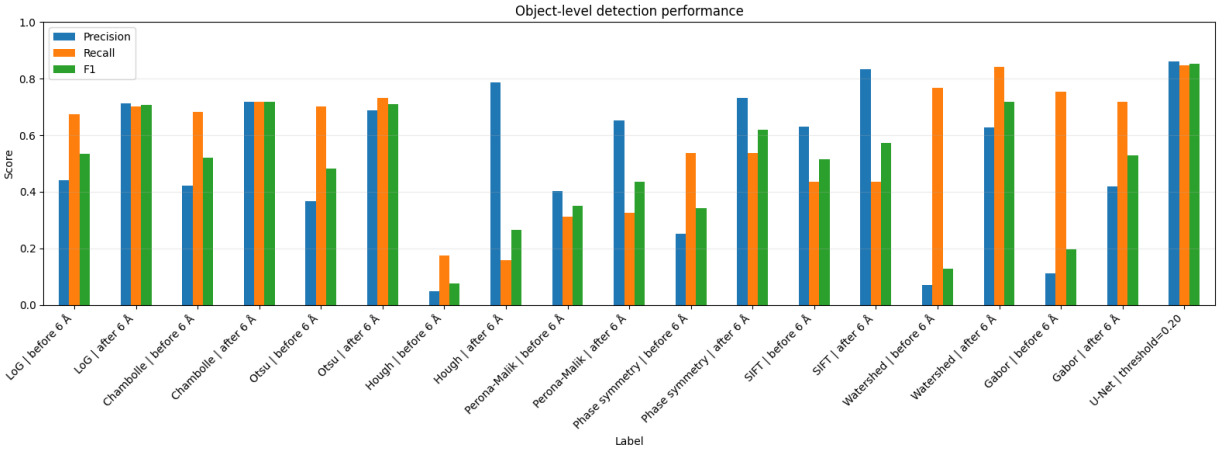

Figure 5: Foreground-aware preprocessing of the AFM image.

8.3 Localisation and parameter errors

Matched QDs can additionally be compared in terms of localisation and measured physical parameters.

For example, for height,

$$e_{h,i} = h_{i,\mathrm{measured}} - h_{i,\mathrm{true}}.$$

The mean absolute error is

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N} |e_i|,$$

while the root-mean-square error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}.$$

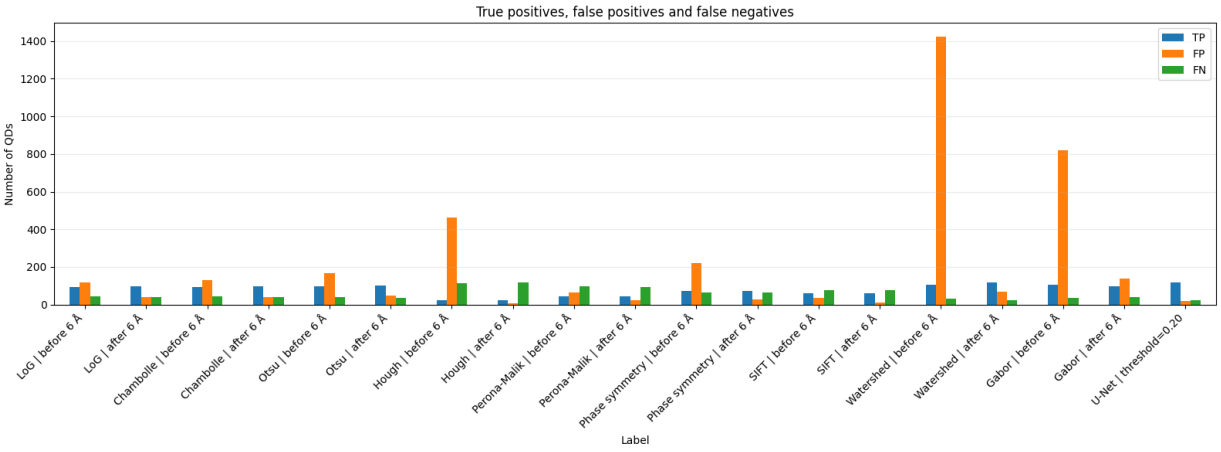

Figure 6: Foreground-aware preprocessing of the AFM image.

9 Machine-Learning Approach

The repository also contains a supervised image-segmentation approach based on a U-Net-like convolutional neural network.

9.1 `Ground-Truth/model.py`

The model-training script loads the paired AFM scans and ground-truth masks, applies the same preprocessing pipeline used elsewhere in the project, and robustly normalises the images.

The scans are divided into training and held-out test data.

Images are subdivided into

$$128 \times 128$$

patches for training.

The implemented network contains an encoder, a bottleneck and a decoder with skip connections.

Convolutional blocks extract increasingly abstract image features while the decoder reconstructs a pixelwise QD probability map.

The final layer uses a sigmoid activation so that each output pixel lies between zero and one.

9.2 Loss function

Training combines binary cross-entropy with a Dice-based loss.

This is useful because QD masks are sparse: most AFM pixels belong to the background rather than to a QD.

9.3 `Ground-Truth/model_prediction.py`

The saved network is evaluated on an unseen scan using the same preprocessing used during training.

The full image is divided into patches, each patch is passed through the network, and the predicted patches are assembled into a complete probability map.

A probability threshold is subsequently applied to obtain a binary QD mask.

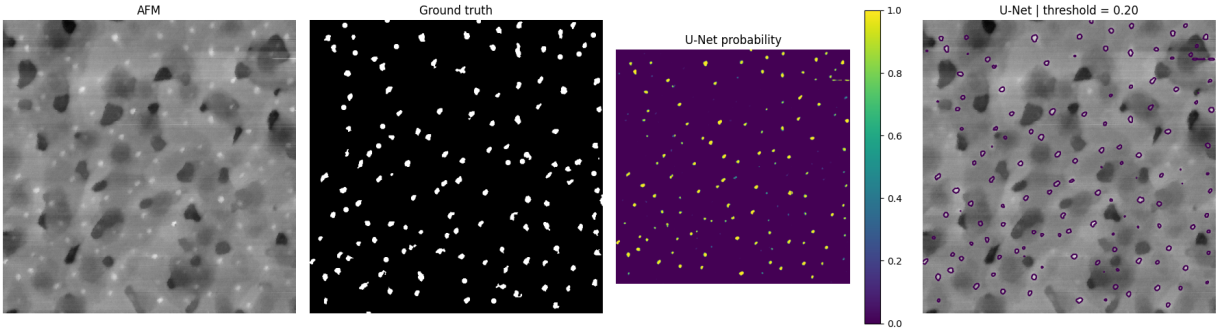

Figure 7: Foreground-aware preprocessing of the AFM image.

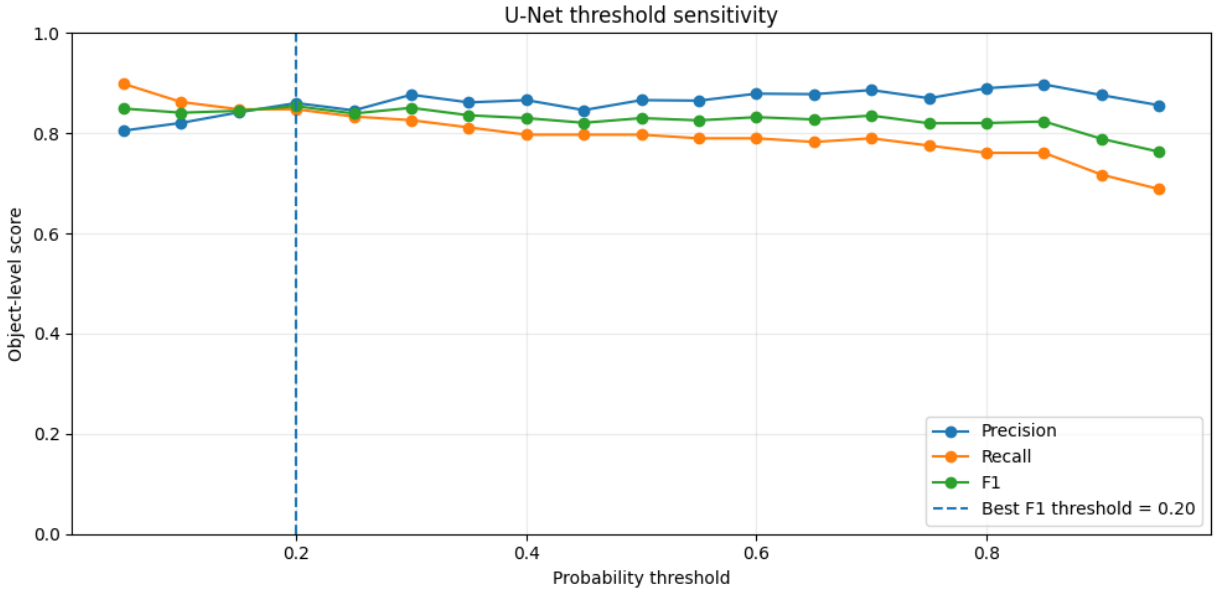

Figure 8: Foreground-aware preprocessing of the AFM image.

10 Height and FWHM Validation

The directory

`Ground-Truth/Height-HWFM/`

contains manually measured QD height and FWHM values for selected samples.

These measurements are used to determine whether the automated parameter-estimation procedure reproduces manually measured physical QD dimensions.

At present this directory contains reference CSV data for several sample conditions, the Python error-analysis script, a human-readable Excel comparison table and generated error figures.

For each matched QD,

$$e_h = h_{\text{automatic}} - h_{\text{manual}}$$

and

$$e_{\text{FWHM}} = \text{FWHM}_{\text{automatic}} - \text{FWHM}_{\text{manual}}.$$

The resulting error distribution is important in addition to the MAE and RMSE.

A distribution centred away from zero indicates systematic bias, while a broad distribution centred near zero indicates low bias but poor precision.

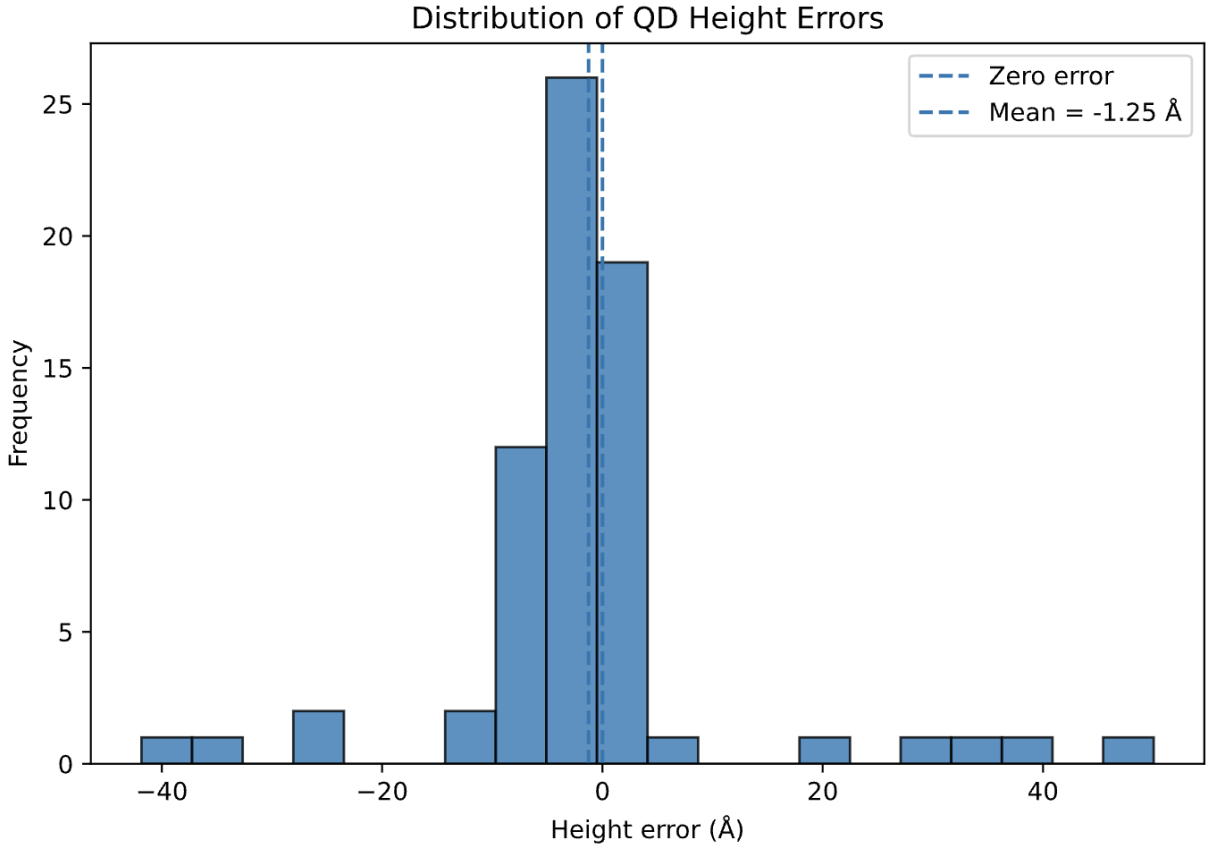

Figure 9: Foreground-aware preprocessing of the AFM image.

11 Analysis Notebooks

Several notebooks are retained for interactive investigation.

11.1 `npy-data-TRANSFORMS.ipynb`

Used for visualising and experimenting with transformations and preprocessing applied to NumPy AFM data.

11.2 `npy-data-QD-DETECTION.ipynb`

Used for interactive testing of QD detection methods and detector parameters.

11.3 `npy-data-QD-ANALYSIS.ipynb`

Used for subsequent analysis of detected QDs and their measured properties.

11.4 `method-comparison.ipynb`

Provides a convenient environment for comparing the behaviour and performance of different detection methods on the same data.

12 Graphical User Interface

The top-level `QD-gui.py` provides an interactive interface around the analysis pipeline.

The GUI is intended to make the processing results directly inspectable rather than requiring every operation to be performed through a notebook or command-line script.

This is particularly useful for checking individual QD detections, reviewing physical measurements and understanding failure cases.

13 Current Project Architecture

The code can therefore be viewed as five connected layers:

1. **Data layer**

   Raw AFM measurements are converted into two-dimensional NumPy height arrays.

2. **Preprocessing layer**

   Large-scale background and scanner artefacts are removed while attempting to preserve physical QD morphology.

3. **Detection layer**

   Multiple classical algorithms and a neural network estimate which structures correspond to QDs.

4. **Physical measurement layer**

   Detected QDs are assigned positions and morphological parameters such as local height, radius, area and FWHM.

5. **Evaluation layer**

   Detection masks and individual QDs are compared against manually curated ground truth using segmentation, object-detection, localisation and parameter-estimation metrics.

14 Questions Currently Being Investigated

The project is currently concerned with several related questions:

- Which preprocessing operations improve QD detection without distorting the physical height and width of the dots? How to quantify the best preprocessing technique?
- Which classical detection algorithm provides the strongest baseline?
- Can combinations of complementary classical algorithms outperform individual detectors?
- How accurately can QD height and FWHM be reproduced relative to manual measurements?

- Are remaining parameter errors random, or do they show systematic dependence on QD size, morphology or local background?

15 Future Work

Possible next stages include

- increasing the number and diversity of manually labelled AFM scans;
- improving object matching and physical-parameter comparison;
- analysing error as a function of QD height, radius and morphology;
- testing more advanced segmentation networks or hybrid classical/learned methods;
- producing a fully automated pipeline from microscope data to final QD statistics.

16 Summary

The project has developed from a simple AFM thresholding problem into a complete QD-analysis framework.

The current repository contains tools for raw-data conversion, robust AFM preprocessing, manual ground-truth construction, multiple classical detection strategies, supervised neural-network segmentation, object-level evaluation and quantitative validation of physical QD measurements.

The central aim is therefore not merely to maximise a segmentation score, but to obtain a detection system that is simultaneously accurate, physically meaningful and capable of producing reliable quantitative measurements of individual quantum dots.